

# ARM Assembly Simulator

State of the Art & Related Work

---

## 1. Introduction

Assembly simulators are educational tools that bridge the gap between high-level programming and bare-metal hardware. They allow students and developers to observe the complete instruction pipeline — from source text to binary execution — without access to physical hardware. This document surveys the current state of the art in assembly simulation, identifies five existing tools with significant relevance to this project, and positions the ARM Assembly Simulator relative to those tools.

---

## 2. State of the Art

### 2.1 Educational CPU simulators — current landscape

Modern educational simulators have evolved significantly over the past decade. The dominant trends are:

- Browser-based interfaces — eliminating the need for local installation, lowering the barrier for students.
- Visual pipeline representation — animating the fetch–decode–execute cycle to make abstract concepts concrete.
- ISA variety — ARM (ARMv7/ARMv8), MIPS, and RISC-V dominate, but custom simplified ISAs are increasingly common for introductory courses.
- Step-by-step debuggers — single-step execution with register diff highlighting has become a standard expectation.
- Open source — most academic simulators are now open source, enabling peer review and extension.

### 2.2 Encoding correctness and flag semantics

A recurring issue in educational simulators is simplified or incorrect flag handling. Many tools implement N and Z correctly but omit or misimplement the carry (C) and overflow (V) flags. The ARM Architecture Reference Manual defines subtraction carry as a no-borrow condition ( $C = rn \geq val2$ ), which differs from addition carry and is frequently implemented incorrectly. Correctly implementing all four flags is necessary for conditional branching to work on all edge cases.

### 2.3 Memory model fidelity

Early simulators maintained separate instruction and data memories — a simplification that does not reflect real hardware. Current best practice uses a unified flat address space with distinct address ranges for code and data. This approach correctly models how real CPUs share a single address bus and makes the simulator more useful for teaching stack behavior and data alignment.

---

## 3. Related Tools

### 3.1 LMC — Little Man Computer (Peter Higginson)

URL: <https://peterhigginson.co.uk/lmc/>

#### Overview

LMC is a browser-based simulator of the Little Man Computer model — a highly simplified von Neumann architecture with a single accumulator, 100 memory locations, and a minimal instruction set (ADD, SUB, STA, LDA, BRA, BRZ, BRP, INP, OUT, HLT). It was created by Peter Higginson and is widely used in UK secondary schools and undergraduate introductory courses. The simulator animates the full fetch–decode–execute cycle with moving data blobs, making the pipeline visible and tangible for beginners.

#### Similarities with ARM Assembly Simulator

- Both are browser-based with no installation required.
- Both expose memory contents and step through the fetch–decode–execute cycle.
- Both support INP (input) and OUT (output) system instructions and HLT for halting.
- Both use a two-pass assembler: labels are collected first, then instructions are encoded.

#### Differences

- ISA complexity: LMC has only 9 instructions and a single accumulator. This project has 37 mnemonics, 16 general-purpose registers, and a full ALU with four condition flags (N, Z, C, V).
- Word size: LMC uses a 3-digit decimal (10-bit effective) word. This project uses 32-bit binary encoding faithful to ARM.
- Branching: LMC has only unconditional, zero, and positive branches. This project has 7 conditional branch types (BEQ, BNE, BLT, BGT, BGE, BLE, B) based on full flag semantics.
- No registers: LMC has no general-purpose registers. This project has R0–R15 with SP, LR, and PC conventions.
- Pseudo-instructions: LMC has none. This project adds INC, DEC, CLR, LSL, LSR, MOD, SWAP, and LOOP.

### 3.2 RISC Simulator (Peter Higginson)

URL: <https://peterhigginson.co.uk/RISC/>

#### Overview

The RISC Simulator is an evolution of LMC by the same author, targeting a 16-bit ARM-inspired ISA with 8 general-purpose registers (R0–R7), condition flags (N, Z, C, V), stack operations (PSH/POP), subroutine calls (JMS/RET), and a much richer instruction set including ADD, SUB, CMP, MOV, AND, ORR, XOR, MVN, MUL, DIV, MOD, LSL, LSR, LDR/STR, and all ARM-standard conditional branches. It retains the animated pipeline visualisation from LMC and adds a binary memory view and instruction counter.

### Similarities with ARM Assembly Simulator

- Both use an ARM-inspired ISA with identical or near-identical mnemonics: MOV, ADD, SUB, CMP, AND, ORR, MVN, MUL, DIV, MOD, LSL, LSR, LDR, STR, PSH, POP, JMS, RET, HLT, INP, OUT, and all 7 ARM conditional branch codes.
- Both implement all four condition flags (N, Z, C, V) with standard ARM semantics.
- Both use a full-descending stack with SP (R13 here, R7/SP there).
- Both are browser-based, no installation required.
- Both support labels and a two-pass assembler.

### Differences

- Word size: RISC Simulator uses 16-bit words. This project uses 32-bit, faithful to the ARM binary encoding format including the rotate-form immediate field.
- Register count: RISC has 8 registers (R0–R7). This project has 16 (R0–R15).
- Encoding fidelity: RISC Simulator uses a custom 16-bit encoding. This project implements the actual ARM 32-bit encoding (cond|opcode|S|Rn|Rd|operand2) including the rotate-form immediate, multiply signature bits, and load/store field layout.
- Pseudo-instruction semantics: RISC expands pseudos with PUSH/POP memory spills. This project expands all pseudos at assemble time using a scratch register, emitting zero stack instructions.
- Pipeline animation: RISC Simulator animates data movement through the CPU schematic. This project shows static memory and register state after execution.
- Testing: RISC Simulator ships with no automated test suite. This project ships with 101 pytest unit tests.

## 3.3 CPULATOR — University of Toronto

URL: <https://cpulator.01xz.net/?sys=arm>

### Overview

CPULATOR is a full-system ARMv7 and MIPS simulator developed at the University of Toronto, widely used in undergraduate computer organisation courses. It runs entirely in the browser using WebAssembly and simulates not only the CPU but also peripherals, interrupts, and memory-mapped I/O.

### Similarities with ARM Assembly Simulator

- Both target students learning low-level programming for the first time.
- Both implement the ARM-like instruction set with condition flags (N, Z, C, V) and conditional branching.
- Both expose registers and memory state after execution.
- Both support a browser interface alongside offline use.

### Differences

- ISA completeness: CPULATOR implements the full ARMv7 ISA including Thumb, NEON, and FPU. This project implements a curated educational subset with custom pseudo-instructions not present in real ARM.
- Scope: CPULATOR simulates a full SoC with DMA, timers, and VGA output. This project targets the CPU pipeline specifically.

- Pseudo-instructions: CPULATOR does not support custom pseudo-instructions. This project adds INC, DEC, CLR, LSL, LSR, MOD, SWAP, and LOOP as assembler-level conveniences.
- Debugger: CPULATOR has a mature step-debugger with memory watches. This project's step-debugger is a planned feature.
- Memory model: CPULATOR uses a segmented hardware-accurate memory map. This project uses a simpler unified flat address space split into instruction and data halves.

### 3.4 Venus — RISC-V Simulator (Berkeley)

URL: <https://venus.cs61c.org>

#### Overview

Venus is an open-source RISC-V simulator created for Berkeley's CS 61C course. It runs in the browser, provides a Monaco-based code editor with syntax highlighting, a full register file view, a memory hex dump with address labels, and single-step execution with change highlighting.

#### Similarities with ARM Assembly Simulator

- Both are browser-based with a code editor on the left and state visualisation on the right.
- Both implement a two-pass assembler: label collection followed by instruction encoding.
- Both expose instruction memory and data memory separately in the UI.
- Both are open source and targeted at undergraduate-level computer architecture courses.

#### Differences

- ISA: Venus implements RISC-V RV32I. This project implements a custom ARM-inspired ISA with rotate-form immediates and ARM condition codes.
- Editor: Venus uses Monaco Editor with full syntax highlighting. This project uses a textarea with line numbers.
- Pseudo-instructions: Venus supports standard RISC-V pseudos (li, la, mv). This project introduces domain-specific pseudos (LOOP, MOD, SWAP) with no memory spill.
- Step debugger: Venus has mature step-debug with register diff colouring. This project has step-debug planned but not yet implemented.
- Deployment: Venus is a static SPA. This project uses a Flask backend (Python assembler/CPU) deployed on Render.

### 3.5 Ripes — Visual RISC-V Pipeline Simulator

URL: <https://github.com/mortbopet/Ripes>

#### Overview

Ripes is an open-source graphical processor simulator and assembly editor built around the RISC-V ISA. Created by Morten Borup Petersen at DTU and used in universities worldwide, it provides an interactive visual representation of a 5-stage pipeline (IF/ID/EX/MEM/WB), showing forwarding paths, hazard detection, and stall cycles in real time. It also includes a cache simulator, C/C++ compilation support via GCC integration, and multiple microarchitectural models from single-cycle to pipelined.

### Similarities with ARM Assembly Simulator

- Both provide a code editor alongside memory and register state visualisation.
- Both are open source and developed for university-level computer architecture education.
- Both support step-by-step execution and expose the register file after each instruction.
- Both implement a two-pass assembler with label resolution.

### Differences

- Pipeline model: Ripes visually simulates the full 5-stage RISC pipeline with forwarding and hazard detection. This project simulates a single-cycle sequential model with no pipeline stages.
- ISA: Ripes implements RISC-V RV32I/M. This project implements a custom ARM-inspired ISA.
- Platform: Ripes is a compiled Qt/C++ desktop application (Windows and Linux). This project runs entirely in any browser with no installation.
- Cache simulation: Ripes includes a configurable cache simulator. This project has no cache model.
- Backend language: Ripes is implemented in C++. This project is implemented in Python, making the source more accessible to students reading the simulator code itself.

---

## 4. Comparison Table

| Tool         | Type         | Lang      | Open src | ISA             | Debugger     | Web UI | Notes                       |
|--------------|--------------|-----------|----------|-----------------|--------------|--------|-----------------------------|
| LMC          | CPU sim      | JS        | Yes      | Custom 9-instr  | Animated FDE | Yes    | UK schools, FDE animation   |
| RISC Sim     | CPU sim      | JS        | No       | ARM-like 16-bit | Animated FDE | Yes    | ARM mnemonics, 16-bit words |
| CPUlator     | Full SoC     | C/WASM    | Yes      | ARMv7/MIPS      | Full         | Yes    | Peripherals, interrupts     |
| Venus        | CPU sim      | Kotlin/JS | Yes      | RISC-V RV32     | Step         | Yes    | Monaco editor, CS61C        |
| Ripes        | Pipeline sim | C++       | Yes      | RISC-V RV32     | Step+visual  | No     | 5-stage pipeline, cache     |
| This project | CPU sim      | Python    | Yes      | ARM-like 32-bit | Planned      | Yes    | Custom pseudos, 101 tests   |

SoC = System on Chip | FDE = Fetch–Decode–Execute | WASM = WebAssembly

---

## 5. Project Positioning

The ARM Assembly Simulator occupies a distinct niche among the five tools above:

- Custom pedagogical ISA with 32-bit ARM encoding: unlike LMC and the RISC Simulator which use simplified or 16-bit custom encodings, this project implements the actual 32-bit ARM field layout (cond|opcode|S|Rn|Rd|operand2, rotate-form immediates, multiply signature bits). Unlike CPULATOR and Venus/Ripes which use standardised ISAs, it also introduces domain-specific pseudo-instructions (LOOP, MOD, SWAP) not found in real ARM.
  - No-spill pseudo-instruction expansion: all pseudo-ops are expanded at assemble time using a scratch register convention, emitting zero stack instructions. The RISC Simulator expands pseudos with memory spills; the other tools do not support equivalent custom pseudos.
  - Unified memory model: unlike many educational simulators that maintain separate instruction and data memories, this project uses a single flat address space split by address range, which is architecturally more accurate.
  - Automated test coverage: 101 pytest unit tests covering every instruction class, all four condition flags, assembler error paths, and end-to-end programs. None of the five comparison tools ship with an automated test suite.
  - Full pipeline transparency in Python: assembler, encoder, decoder, and CPU are distinct Python modules. Students can read and modify the entire simulator in the same language they learn first, lowering the barrier compared to C++ (Ripes) or WebAssembly (CPULATOR).
- 

## 6. References

**LMC (Peter Higginson):** <https://peterhigginson.co.uk/lmc/>

**RISC Simulator (Peter Higginson):** <https://peterhigginson.co.uk/RISC/>

**CPULATOR (University of Toronto):** <https://cpulator.01xz.net/?sys=arm>

**Venus RISC-V Simulator (Berkeley):** <https://venus.cs61c.org>

**Ripes — Visual RISC-V Pipeline Simulator:** <https://github.com/mortbopet/Ripes>

**ARM Architecture Reference Manual (ARMv7-A/R):**  
<https://developer.arm.com/documentation/ddi0406/latest>

**RISC-V Instruction Set Manual:** <https://riscv.org/technical/specifications/>

**Project GitHub Repository:** <https://github.com/Alexia-Crisan/ARM-Assembly-Simulator>